AMS/WMS Backend Information and Flow

1. Project Backend Structure

The project follows a layered backend architecture:

- `D:\ams-wms-platform`: Root project folder
- `D:\ams-wms-platform\backend`: Main backend workspace
- `D:\ams-wms-platform\backend\business-service`: Main Python application service (operational WMS/AMS workflows)
- `D:\ams-wms-platform\backend\auth-service`: Java authentication service
- `D:\ams-wms-platform\backend\common`: Shared utilities

2. Backend Functionality

The `business-service` handles core operations:

- Receiving and inbound flow
- Returns processing
- Notification events
- Gate entry and vehicle arrival flow
- Procurement operations
- Dashboard and operational APIs
- Domain logic and persistence

3. Main Service Technical Details

- **Service Name:** `business-service`
- **Framework:** FastAPI (Python 3.12+)
- **Persistence:** SQLAlchemy async ORM (PostgreSQL)
- **Messaging:** Kafka
- **Cache/Queue:** Redis

- **Auth:** JWT via `auth-service` + JWKS
- **Migrations:** Alembic

4. Runtime API Lifecycle

1. **Startup:** App initialization, Kafka producer, outbox relay, notification consumer.
2. **Request:** Middleware, CORS, exception handling.
3. **Auth:** Validation via external service.
4. **Dispatch:** Routing to modules (receiving, returns, gate, etc.).
5. **Business Logic:** Domain objects and use cases.
6. **Persistence:** UoW/Repository writes to PostgreSQL.
7. **Outbox/Event:** Event written to DB, relayed to Kafka.
8. **Response:** Final HTTP result.

5. Receiving Flow Pattern

A typical operational flow follows this pattern: `Frontend -> Router -> Auth Validation -> Business Logic -> Persistence -> Outbox Event -> Kafka -> Notification Consumer`.

6. Infrastructure & Deployment

Services are orchestrated via `docker-compose.yml`:

- Start: `docker compose up -d`
- DBs: PostgreSQL (5433), Redis (6379)
- Services: `business-service` (8000), `auth-service` (8080), Kafka (9092)